

Ranking Functions in PostgreSQL: ROW_NUMBER, RANK, DENSE_RANK, NTILE

Table of Contents

1. [Learning Objectives](#)
 2. [Theory: Ranking in SQL](#)
 3. [ROW_NUMBER](#)
 4. [RANK](#)
 5. [DENSE_RANK](#)
 6. [NTILE](#)
 7. [Comparing All Four Functions](#)
 8. [Top-N Per Group Pattern](#)
 9. [Deduplication with ROW_NUMBER](#)
 10. [Pagination with ROW_NUMBER](#)
 11. [ASCII Visual Diagrams](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Hands-on Exercises](#)
 17. [Advanced Notes](#)
 18. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Distinguish ROW_NUMBER, RANK, DENSE_RANK, and NTILE
 - Implement top-N per group queries using ranking functions
 - Use ROW_NUMBER for deduplication and pagination
 - Apply NTILE for percentile bucketing and A/B test assignment
 - Solve the most common ranking interview problems
-

Theory: Ranking in SQL

Ranking functions assign a rank number to each row within a partition based on the ORDER BY expression. They differ in how they handle ties (equal ORDER BY values).

For values: [100, 100, 90, 80, 80, 70]

ROW_NUMBER:	[1, 2, 3, 4, 5, 6]	– always unique, arbitrary tie-break
RANK:	[1, 1, 3, 4, 4, 6]	– ties share rank, gaps after ties
DENSE_RANK:	[1, 1, 2, 3, 3, 4]	– ties share rank, NO gaps
NTILE(3):	[1, 1, 2, 2, 3, 3]	– divides into N equal buckets

ROW_NUMBER

Assigns a unique sequential integer to each row within the partition. No two rows ever share the same number. The tie-breaking order within equal values is non-deterministic unless a fully deterministic ORDER BY is used.

Syntax

```
ROW_NUMBER() OVER (  
    [PARTITION BY partition_cols]  
    ORDER BY order_cols  
)
```

Examples

```
-- Example 1: Basic ROW_NUMBER – global rank by salary  
SELECT  
    emp_name,  
    department,  
    salary,  
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS global_rank  
FROM emp_salary  
ORDER BY global_rank;  
  
-- Example 2: ROW_NUMBER per department  
SELECT  
    emp_name,  
    department,  
    salary,  
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank  
FROM emp_salary  
ORDER BY department, dept_rank;  
  
-- Example 3: Top-1 per department (highest earner)  
SELECT emp_name, department, salary  
FROM (  
    SELECT  
        emp_name,  
        department,  
        salary,  
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn  
    FROM emp_salary  
    ) AS ranked  
WHERE rn = 1;  
  
-- Example 4: Bottom-2 per department (lowest earners)  
SELECT emp_name, department, salary  
FROM (  
    SELECT  
        emp_name,
```

```

        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary ASC) AS rn
    FROM emp_salary
) AS ranked
WHERE rn <= 2
ORDER BY department, salary;

-- Example 5: ROW_NUMBER with secondary sort for deterministic ordering
SELECT
    emp_name,
    department,
    salary,
    hire_date,
    ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY salary DESC, hire_date ASC -- tie-break by hire date
    ) AS stable_rank
FROM emp_salary
ORDER BY department, stable_rank;

```

RANK

Assigns a rank to each row. Tied rows receive the same rank, and the next rank skips numbers equal to the count of ties.

Examples

```

-- Example 6: RANK – salary rank with gaps for ties
SELECT
    emp_name,
    department,
    salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank
FROM emp_salary
ORDER BY department, salary_rank;

-- Example 7: Global rank using RANK
SELECT
    emp_name,
    salary,
    RANK() OVER (ORDER BY salary DESC) AS salary_rank,
    RANK() OVER (ORDER BY salary ASC) AS lowest_salary_rank
FROM emp_salary
ORDER BY salary_rank;

-- Example 8: Find all employees tied for first in their department
SELECT emp_name, department, salary
FROM (
    SELECT

```

```

        emp_name,
        department,
        salary,
        RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
    FROM emp_salary
) AS ranked
WHERE rnk = 1
ORDER BY department;

```

DENSE_RANK

Like RANK, but without gaps. Tied rows share a rank, and the next distinct rank is always the next consecutive integer.

Examples

```

-- Example 9: DENSE_RANK – no gaps after ties
SELECT
    emp_name,
    department,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_global
FROM emp_salary
ORDER BY dense_rank_global;

-- Example 10: DENSE_RANK to find the N-th highest salary
-- Find employees with the 2nd highest salary in each dept
SELECT emp_name, department, salary
FROM (
    SELECT
        emp_name,
        department,
        salary,
        DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
    FROM emp_salary
) AS ranked
WHERE dr = 2
ORDER BY department;

-- Example 11: All three ranking functions side by side
SELECT
    emp_name,
    department,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
FROM emp_salary
ORDER BY department, salary DESC;

```

NTILE

Divides rows within a partition into N buckets (tiles) as evenly as possible. Rows in earlier buckets may have one more row than later buckets when rows cannot be divided evenly.

Examples

```
-- Example 12: NTILE(4) – quartiles by salary
SELECT
    emp_name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile
FROM emp_salary
ORDER BY salary_quartile, salary DESC;

-- Example 13: NTILE per department – performance buckets
SELECT
    emp_name,
    department,
    salary,
    NTILE(3) OVER (PARTITION BY department ORDER BY salary DESC) AS perf_bucket,
    CASE NTILE(3) OVER (PARTITION BY department ORDER BY salary DESC)
        WHEN 1 THEN 'Top Earner'
        WHEN 2 THEN 'Mid Range'
        WHEN 3 THEN 'Bottom Third'
    END AS bucket_label
FROM emp_salary
ORDER BY department, salary DESC;

-- Example 14: NTILE for A/B test assignment
SELECT
    emp_name,
    salary,
    CASE NTILE(2) OVER (ORDER BY emp_id) -- random-ish split by ID
        WHEN 1 THEN 'Control'
        WHEN 2 THEN 'Treatment'
    END AS test_group
FROM emp_salary;

-- Example 15: Revenue percentiles with NTILE
SELECT
    department,
    month_date,
    revenue,
    NTILE(100) OVER (ORDER BY revenue) AS percentile_rank
FROM monthly_revenue
ORDER BY percentile_rank DESC;
```

Comparing All Four Functions

ASCII Diagram

Data (by salary DESC): Alice=105000, Bob=95000, Carol=87000, David=72000, Eve=68000
(two rows with equal salary, if Carol and Bob were both 95000)

	salary	ROW_NUMBER	RANK	DENSE_RANK	NTILE(3)	
Alice	105000	1	1	1	1	
Bob	95000	2	2	2	1	
Carol	95000	3	2	2	2	← tie with Bob
David	72000	4	4	3	2	← gap in RANK
Eve	68000	5	5	4	3	
Frank	68000	6	5	4	3	← tie with Eve

ROW_NUMBER:	always unique, arbitrary tie-break	[1,2,3,4,5,6]
RANK:	ties get same number, gap after	[1,2,2,4,5,5]
DENSE_RANK:	ties get same number, no gap	[1,2,2,3,4,4]
NTILE(3):	divides into 3 buckets (2+2+2)	[1,1,2,2,3,3]

Top-N Per Group Pattern

One of the most common interview problems. The standard solution uses ROW_NUMBER in a subquery.

```
-- Example 16: Top-3 earners per department
SELECT emp_name, department, salary, dept_rank
FROM (
    SELECT
        emp_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS
dept_rank
    FROM emp_salary
) AS ranked
WHERE dept_rank <= 3
ORDER BY department, dept_rank;

-- With RANK (allows ties in top-3):
SELECT emp_name, department, salary, dept_rank
FROM (
    SELECT
        emp_name,
        department,
        salary,
        RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
    FROM emp_salary
) AS ranked
WHERE dept_rank <= 3
ORDER BY department, dept_rank;
```

Deduplication with ROW_NUMBER

ROW_NUMBER is the canonical tool for deduplication (keeping the most recent, largest, or first record from a group of duplicates).

```
-- Example 17: Deduplication – keep latest order per customer
SELECT customer_id, order_id, order_date, total_amount
FROM (
    SELECT
        customer_id,
        order_id,
        order_date,
        total_amount,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date DESC, order_id DESC
        ) AS rn
    FROM orders
) AS deduped
WHERE rn = 1
ORDER BY customer_id;

-- Example 18: Delete duplicates using ROW_NUMBER in a CTE
WITH duplicates AS (
    SELECT
        sale_id,
        ROW_NUMBER() OVER (
            PARTITION BY rep_name, sale_date, product, amount
            ORDER BY sale_id
        ) AS rn
    FROM sales
)
DELETE FROM sales
WHERE sale_id IN (SELECT sale_id FROM duplicates WHERE rn > 1);
```

Pagination with ROW_NUMBER

```
-- Example 19: Keyset pagination – page 2, 5 rows per page
SELECT emp_name, department, salary
FROM (
    SELECT
        emp_name,
        department,
        salary,
        ROW_NUMBER() OVER (ORDER BY salary DESC, emp_id) AS rn
    FROM emp_salary
) AS paged
WHERE rn BETWEEN 6 AND 10; -- page 2: rows 6-10
```

Common Mistakes

Mistake 1: Forgetting ORDER BY inside OVER — ROW_NUMBER without order is non-deterministic

```
-- Technically valid but order is undefined (non-deterministic):  
ROW_NUMBER() OVER () -- random order without ORDER BY  
  
-- Always specify ORDER BY for ranking functions:  
ROW_NUMBER() OVER (ORDER BY salary DESC)
```

Mistake 2: Using RANK where DENSE_RANK was intended

```
-- If you want "2nd highest salary", RANK skips to 3 after two ties at 1  
-- Use DENSE_RANK for "Nth distinct value" patterns
```

Mistake 3: Filtering on window function result in WHERE

```
-- WRONG:  
SELECT * FROM emp_salary WHERE ROW_NUMBER() OVER (ORDER BY salary) = 1;  
  
-- CORRECT:  
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (ORDER BY salary) AS rn FROM emp_salary)  
t  
WHERE rn = 1;
```

Mistake 4: NTILE uneven distribution

```
-- NTILE(3) on 7 rows: buckets get [3, 2, 2] rows  
-- Earlier buckets are larger when rows don't divide evenly  
-- Don't assume exactly N/buckets rows per tile
```

Best Practices

1. Use ROW_NUMBER for deduplication and exact top-1 per group.
2. Use RANK when you want ties to share the same rank (Olympic-style).
3. Use DENSE_RANK when you want the Nth distinct value without gaps.
4. Use NTILE for even distribution into buckets (percentiles, A/B testing).
5. Always include a tie-breaking column in ORDER BY for deterministic results.
6. Wrap ranking queries in a CTE or subquery before filtering on the rank.

Performance Considerations


```
-- Ranking functions require sorting the input
-- Index on PARTITION BY + ORDER BY columns helps:
CREATE INDEX idx_emp_dept_salary ON emp_salary(department, salary DESC);

-- Verify the plan uses an index scan + WindowAgg (not a full sort):
EXPLAIN ANALYZE
SELECT emp_name, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS
rn
FROM emp_salary;

-- For deduplication patterns, consider DISTINCT ON for simpler cases:
-- DISTINCT ON is often faster for exact top-1 per group in PostgreSQL
SELECT DISTINCT ON (department) emp_name, department, salary
FROM emp_salary
ORDER BY department, salary DESC;
-- Equivalent to ROW_NUMBER() WHERE rn = 1, but more efficient
```

Interview Questions & Answers

Q1: What is the difference between RANK and DENSE_RANK?

A: Both assign the same rank to tied rows. RANK leaves gaps after ties — if two rows tie at rank 1, the next rank is 3 (not 2). DENSE_RANK never leaves gaps — after two rows at rank 1, the next distinct rank is 2. Use DENSE_RANK when you want to find the "Nth distinct value."

Q2: When would you use ROW_NUMBER vs RANK?

A: Use ROW_NUMBER when you need exactly one row per rank (no ties allowed) — for top-1 selection, deduplication, and pagination. Use RANK when ties should share the same rank — for competitive rankings where multiple entities can share a position.

Q3: How do you find the employee with the second-highest salary using window functions?

A:

```
SELECT emp_name, salary FROM (
    SELECT emp_name, salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS dr
    FROM emp_salary
) t WHERE dr = 2;
```

DENSE_RANK is needed because multiple employees might share the highest salary.

Q4: How do you write a "Top-N per Group" query?

A: Use ROW_NUMBER (or RANK/DENSE_RANK if ties should be included) in a subquery, then filter:

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY group_col ORDER BY order_col DESC) AS
rn
```

```
FROM table
) t WHERE rn <= N;
```

Q5: What does NTILE do with an uneven number of rows?

A: NTILE distributes rows as evenly as possible. When rows don't divide evenly, earlier buckets receive one extra row. For NTILE(4) on 7 rows, the distribution is [2, 2, 2, 1].

Q6: How do you delete duplicate rows using ROW_NUMBER?

A: Use ROW_NUMBER in a CTE to identify duplicates (rn > 1), then DELETE matching row IDs from the base table.

Q7: What is the difference between ROW_NUMBER() and OFFSET for pagination?

A: OFFSET-based pagination is simpler but inefficient for large pages (PostgreSQL must scan and discard OFFSET rows). ROW_NUMBER-based pagination with a WHERE clause on the row number is similar in efficiency. Keyset (cursor-based) pagination is the most efficient for deep pagination.

Q8: Can you use RANK without PARTITION BY?

A: Yes. Without PARTITION BY, the entire result set is one window, and RANK assigns ranks globally across all rows.

Hands-on Exercises

Exercise 1: Top 2 Earners per Department

Use ROW_NUMBER to find the top 2 highest-paid employees in each department.

```
-- Solution:
SELECT emp_name, department, salary, dept_rank
FROM (
    SELECT
        emp_name,
        department,
        salary,
        ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC, emp_id) AS
dept_rank
    FROM emp_salary
) AS ranked
WHERE dept_rank <= 2
ORDER BY department, dept_rank;
```

Exercise 2: Salary Quartile Assignment

Assign each employee to a salary quartile (1=top, 4=bottom) globally.

```
-- Solution:
SELECT
    emp_name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) AS quartile,
```

```

CASE NTILE(4) OVER (ORDER BY salary DESC)
  WHEN 1 THEN 'Top 25%'
  WHEN 2 THEN '25-50%'
  WHEN 3 THEN '50-75%'
  WHEN 4 THEN 'Bottom 25%'
END AS quartile_label
FROM emp_salary
ORDER BY salary DESC;

```

Exercise 3: Compare Ranking Functions

Show all four ranking functions side by side for the emp_salary table, ordered by salary descending.

```

-- Solution:
SELECT
  emp_name,
  department,
  salary,
  ROW_NUMBER() OVER w AS row_num,
  RANK() OVER w AS rnk,
  DENSE_RANK() OVER w AS dense_rnk,
  NTILE(4) OVER w AS quartile
FROM emp_salary
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC;

```

Exercise 4: Deduplication

In the orders table, keep only the most recent order per customer (highest order_id as tie-break). Delete the rest.

```

-- Solution (preview with SELECT first):
SELECT order_id, customer_id, order_date, total_amount
FROM (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC,
order_id DESC) AS rn
  FROM orders
) AS t
WHERE rn > 1; -- these would be deleted

-- DELETE version:
WITH to_delete AS (
  SELECT order_id,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC,
order_id DESC) AS rn
  FROM orders
)
DELETE FROM orders WHERE order_id IN (SELECT order_id FROM to_delete WHERE rn > 1);

```

Advanced Notes

CUME_DIST and PERCENT_RANK

Related ranking functions for statistical analysis:

```
SELECT
  emp_name,
  salary,
  PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank,
  -- 0.0 for lowest, 1.0 for highest
  CUME_DIST() OVER (ORDER BY salary) AS cume_dist
  -- fraction of rows <= current row (always > 0)
FROM emp_salary
ORDER BY salary;
```

DISTINCT ON (PostgreSQL-specific top-1)

```
-- Faster alternative to ROW_NUMBER() WHERE rn = 1 for simple cases:
SELECT DISTINCT ON (department) emp_name, department, salary
FROM emp_salary
ORDER BY department, salary DESC;
```

Cross-references

- [01_window_functions_intro.md](#) — OVER, PARTITION BY, frame clauses
- [03_analytical_functions.md](#) — LAG, LEAD, FIRST_VALUE for navigation
- [08_interview_window_tasks.md](#) — 20+ interview problems using ranking
- [03_subqueries.md](#) (03_Intermediate_SQL) — subquery pattern to filter on rank